Latent Class Analysis Using MCMC Methods: An Analysis of Burnout and Depression Profiles in the Tech Industry

Gianfilippo Tito*

2026-06-08

1 Introduction

This study is based on an analysis of the open-source dataset *Mental Health & Burnout in Tech Workers*, available on Kaggle. It contains 100,000 observations of tech workers and tracks their mental health, burnout severity, anxiety, depression, and workplace well-being.

The question is: "Is it possible to identify the existence of psychological profiles (latent classes) that are not directly observable among professionals in the tech sector, based on the joint distribution of their burnout and depression scores, using Bayesian analysis via MCMC methods?"

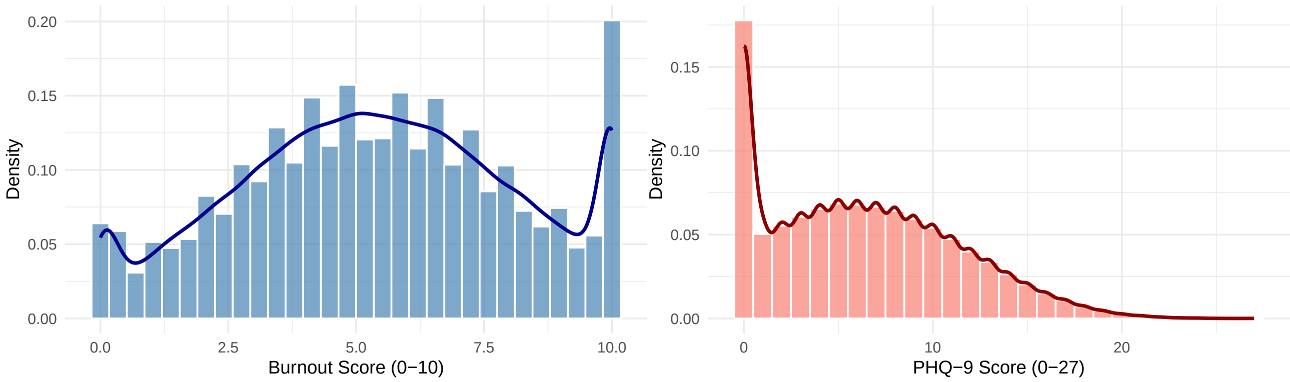
First, an exploratory analysis was conducted to understand the distribution of the key variables underlying our latent class model: the Burnout score and the Depression score.

Table 1: Summary

Burnout Score	Depression Score
Min. : 0.000	Min. : 0.000
1st Qu.: 3.500	1st Qu.: 2.000
Median : 5.400	Median : 6.000
Mean : 5.402	Mean : 6.272
3rd Qu.: 7.400	3rd Qu.:10.000
Max. :10.000	Max. :27.000

Let's proceed with a brief graphical exploration of the joint distribution between these two variables.

*MSc Candidate, Department of Statistics, University of Perugia



(a) Burnout Distribution

(b) Depression Distribution

We can see that the Burnout score is centered around the mean (approximately 5.4), but with a clear range spanning the entire spectrum from 0 to 10. As for the PHQ-9 (Depression) score, we observe a typical right-skewed distribution: most workers cluster around low values, but there is a long tail extending toward higher values (up to 27), indicating a minority of the population with high depression scores.

2 Model and Gibbs Sampler

Define our Gaussian mixture model, the chosen prior distributions, and the *full conditionals* required to implement a Gibbs sampler.

2.1 Model of Data

Let $y_i = (y_{i1}, y_{i2})^\top$ be the vector of continuous observations for individual $i = 1, \dots, n$, where the two components represent the burnout score and the depression score (PHQ-9). We assume that the probability distribution consists of $K = 3$ unobservable latent classes. Let us introduce a latent indicator variable $z_i \in \{1, 2, 3\}$ that specifies whether the i-th individual belongs to a specific class. Given that the individual belongs to class k, we assume that the data follow a bivariate normal distribution:

$$\mathbf{y}_i \mid z_i = k, \mu_k, \mathbf{\Sigma}_k \sim \mathcal{N}_2(\mu_k, \mathbf{\Sigma}_k)$$

The prior probability that an individual belongs to class k is defined by the mixture weights $\lambda_k = P(z_i = k)$, subject to the constraint $\sum_{k=1}^{K} \lambda_k = 1$.

By marginalizing with respect to z_i, the marginal likelihood function for a single observation takes the form of a mixture:

$$p(\mathbf{y}_i \mid \underline{\lambda}, \underline{\mu}, \mathbf{\Sigma}) = \sum_{k=1}^{K} \lambda_k \cdot \phi(\mathbf{y}_i; \mu_k, \mathbf{\Sigma}_k)$$

2.2 Prior Distributions

- Weights of the Mixture ($\underline{\lambda}$):

$$\lambda \sim Dir(\alpha_1, \alpha_2, \alpha_3)$$

where we choose the hyperparameter $\alpha_k = 1$, which reflects the absence of prior information about the proportions of the groups

- Mean (μ_k): Let us adopt an improper "flat" prior over the entire real space:

$$p(\mu_k) \propto 1$$

2.3 Derivation of Posterior Distributions (*Full Conditional*)

The Gibbs Sampler is an MCMC algorithm that works by iteratively updating each block of parameters, sampling them from their posterior distribution conditioned on the other parameters and the auxiliary parameters.

2.3.1 Updating the Latent Variables (z_i)

For each individual, we calculate the *posterior* probability of belonging to class k. Sampling is performed from a categorical distribution:

$$P(z_i = k \mid \mathbf{y}_i, \underline{\lambda}, \underline{\mu}) = \frac{\lambda_k \phi(\mathbf{y}_i; \mu_k, \mathbf{\Sigma}_k)}{\sum_{j=1}^{K} \lambda_j \phi(\mathbf{y}_i; \mu_j, \mathbf{\Sigma}_j)}$$

2.3.2 Updating the Mixture Weights ($\underline{\lambda}$)

Given the current assignments z_i, the number of observations in class k is calculated as $n_k = \sum_{i=1}^{n} \mathbb{I}(z_i = k)$. The *posterior* distribution for λ remains a Dirichlet distribution, updated by counting the class frequencies:

$$\underline{\lambda} \mid \mathbf{z} \sim Dir(\underline{\alpha} + \underline{n})$$

with $\underline{\alpha} = (\alpha_1, \alpha_2, \alpha_3)'$ and $\underline{n} = (n_1, n_2, n_3)'$

2.3.3 Updating Cluster Means (μ_k)

By combining the normal likelihood with the flat prior, the *posterior* distribution for the mean of cluster k turns out to be another multivariate normal distribution. It is centered on the sample mean $\overline{y}_k$ of only the observations assigned to cluster k in the previous step, with the covariance matrix scaled inversely proportional to the cluster size n_k:

$$\mu_k \mid \mathbf{y}, \mathbf{z} \sim \mathcal{N}_2\left(\overline{y}_k, \frac{1}{n_k}\Sigma_k\right)$$

3 Code

```
# Extract 5000 random sample
set.seed(42)
Y = as.matrix(df_clean[sample(1:nrow(df_clean), 5000), ])
n = nrow(Y); d = ncol(Y); k = 3
R = 10000 #iteration
```

```r
# hyperparameters
al = rep(1, k) # prior Dirichlet

# starting values (via K-means)
km = kmeans(Y, centers=k, nstart=10)
mu = km$centers
S = list()
for(u in 1:k) S[[u]] = cov(Y[km$cluster==u, ]) + diag(1e-4, d) #fixed Covariance
la = rep(1/k, k)
z = km$cluster

# Gibbs sampler storage
La = matrix(0, R, k)
Mu = array(0, c(k, d, R))
Z = matrix(0, R, n)

# iterate
for(h in 1:R){
  # update lambda
  nz = table(factor(z, levels=1:k))
  alt = al + nz
  la = as.vector(rdirichlet(1, alt))

  # update mu
  for(u in 1:k){
    if(nz[u] > 0){
      y_bar = colMeans(Y[z==u, , drop=FALSE])
      mu[u,] = rmvnorm(1, y_bar, S[[u]]/nz[u])
    } else {
      mu[u,] = Y[sample(1:n, 1), ]
    }
  }
  # update z
  Pc = matrix(0, n, k)
  for(u in 1:k) Pc[,u] = dmvnorm(Y, mu[u,], S[[u]])

  pm = as.vector(Pc %*% la)
  Pp = (1/pm) * (Pc %*% diag(la))
  Pp[is.na(Pp)] = 1/k
  Pp = sweep(Pp, 1, rowSums(Pp), "/")

  for(i in 1:n) z[i] = sample(1:k, 1, prob=Pp[i,])
```

```
  # label switching
  ind = order(mu[,1])
  if(any(ind != (1:k))){
    la = la[ind]
    mu = mu[ind,]
    S = S[ind]
    z1 = z
    for(u in 1:k) z[z1==ind[u]] = u
  }
  # store parameters
  La[h,] = la
  Mu[,,h] = mu
  Z[h,] = z
}
```

4 Results

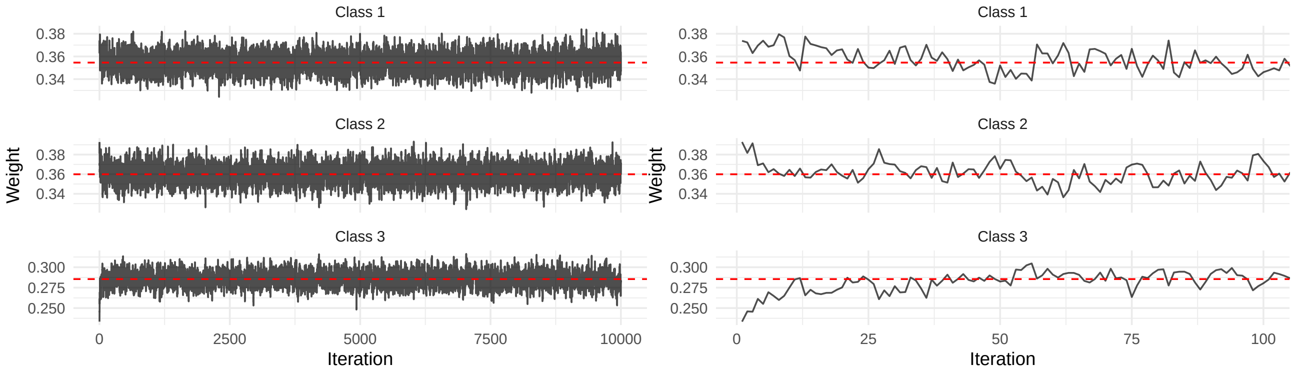
Let's now turn to an analysis of the results shown in Table 2 below. Our algorithm identified three distinct latent profiles within the sample. Class 1, which accounts for 35.5% of professionals, represents a healthy profile, characterized by very low levels of burnout (mean 3.19) and no depressive symptoms (mean PHQ-9 score of 1.20). The largest group is Class 2 (36.0%), which identifies a "Moderate Risk" profile, with intermediate values for both burnout (5.55) and mood (6.54). Finally, Class 3 identifies the most critical clinical segment of the population (28.5%), showing severe burnout (8.17) consistently associated with a high PHQ-9 score (12.28), indicative of a significant level of depression.

Table 2: Posterior estimates (MCMC averages) and diagnostics (ESS) for latent psychological profiles

Latent Profiles	Weights	Burnout Score	PHQ-9 Score	ESS (min)
Class 1	0.355	3.186337	1.199293	2282
Class 2	0.360	5.549191	6.535645	1032
Class 3	0.285	8.171380	12.282528	1411

4.1 Plots

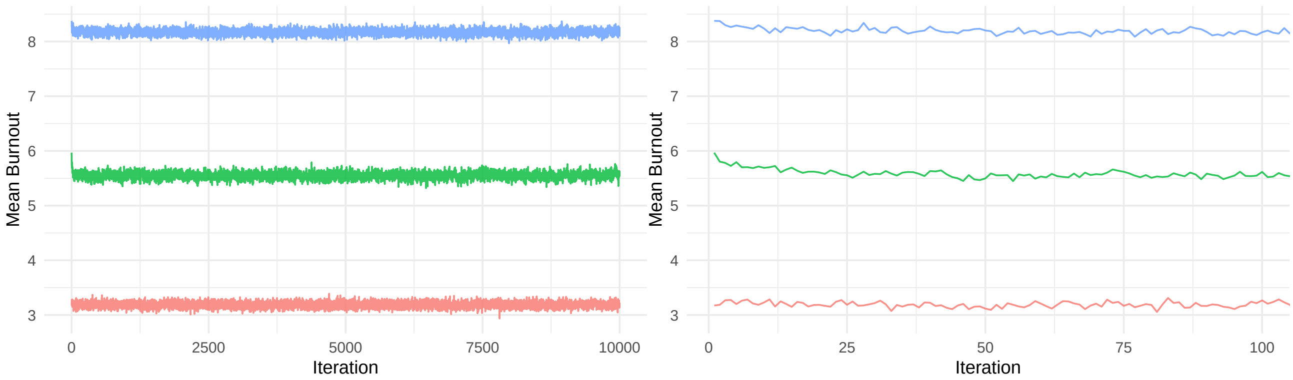
A visual analysis of the traceplots confirms the correct operation and stability of the Gibbs sampler. The complete chains exhibit the typical steady-state behavior, optimally exploring the parameter space without showing any anomalous trends. The initial data reveal an extremely rapid burn-in phase, with the chains converging toward the posterior means within the first 50 iterations.



(a) Full Traceplot (b) 100 iteration

Figure 2: Traceplot Class Weights (Lambda)

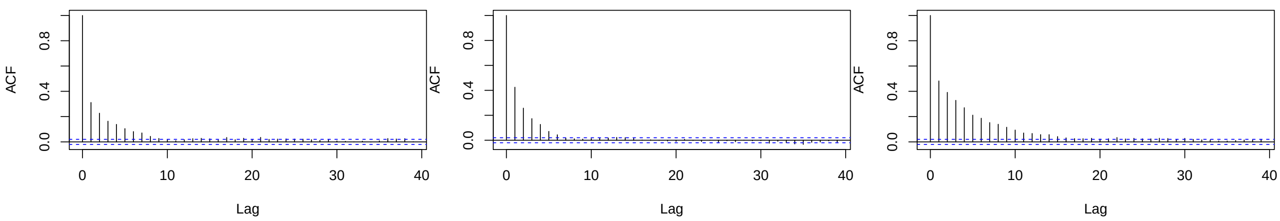
Furthermore, the graph showing the Burnout averages demonstrates that the algorithm successfully handled the inherent problem of label switching: the three sampling bands remain clearly separated and parallel throughout all 10,000 iterations, ensuring perfect class distinguishability.



(a) Full Traceplot (b) 100 iteration

Figure 3: Burnout Means Traceplot

To validate the efficiency of Bayesian inference, the independence of the extracted samples was measured. The autocorrelation function (ACF) decays rapidly to zero within the first 20 lags for all profiles, indicating low sequential dependence among iterations. This visual result is quantitatively supported by the Effective Sample Size (ESS): the minimum values recorded for the three classes (2282, 1032, and 1411, respectively) exceed the critical threshold of 1000. This ensures that the posterior estimates are statistically robust and based on a more than sufficient number of independent samples to draw reliable conclusions.



(a) ACF Class 1 (b) ACF Class 2 (c) ACF Class 3

Figure 4: Autocorrelation Plots